



CIS6003 - Advanced Programming

Assessment WRIT1

Sunrise Dental Clinic

Appointment and Patient Management System

A distributed, three-tier Java application

Student Name: [Your Full Name]

Cardiff Student ID: [stXXXXXXXX]

ICBT Student ID: [CL/BSCSD/XX/XX]

Module Leader: Ms. Priyanga

Academic Year: 2024 / 2025 - Semester 1

Submitted: August 2026

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to everyone who helped me complete this appointment and patient management system for Sunrise Dental Clinic.

First and foremost, I extend my deepest thanks to Ms. Priyanga, my Module Leader for the Advanced Programming module, whose teaching on design patterns, layered architecture and test-driven development shaped almost every decision in this project. The habit of asking "which class should own this rule?" before writing any code came directly from those sessions, and it is the single thing that kept the system tidy as it grew.

I am grateful to ICBT Campus and the School of Technologies for providing an excellent academic environment, and to Cardiff Metropolitan University for the structure of the programme, which pushed me to think about professional standards rather than only about getting the program to run.

I also want to thank the open-source community. This project stands on work I did not have to write myself: the Spring Boot, Hibernate and Flyway projects, the JUnit 5, Mockito and AssertJ maintainers, and the H2 and MySQL teams. Their documentation was often clearer than any textbook.

Finally, I would like to thank my family and my classmates. Explaining a problem out loud to somebody else was, more than once, how I found the answer to it.

Table of Contents

Table of Contents	3
Table of Figures	5
Table of Tables	6
Task A - System Design with UML Diagrams	7
1.1 Understanding the problem.....	7
1.2 Assumptions made	7
1.3 Use Case diagram	8
1.3.1 Choosing between <<include>> and <<extend>>.....	8
1.4 Class diagram.....	8
1.4.1 Multiplicity, navigability, aggregation, composition.....	9
1.5 Sequence diagrams.....	9
1.6 Critical reflection	10
Task B - Design Patterns and Architecture Development	16
2.1 Three-tier architecture.....	16
2.2 A genuinely distributed application with web services	17
2.3 Design patterns applied.....	17
2.3.1 Strategy and Template Method together	18
2.3.2 Chain of Responsibility, and Observer	18
2.3.3 Critical evaluation - what the patterns cost.....	19
2.4 The database.....	19
2.4.1 How double booking is actually prevented.....	19
2.4.2 Advanced database features	20
2.5 Interfaces and validation	20
2.6 Reports that add value.....	21
2.7 Sessions and cookies.....	21
2.8 The system in use.....	24

Task C - Test Plan and Test-Driven Development	30
3.1 Test rationale.....	30
3.2 How test-driven development was used	30
3.3 Test plan.....	31
3.4 Test data	31
3.5 Test automation and results	32
3.6 What the testing actually found	33
3.7 Traceability	33
3.8 Evaluation and lessons learned	34
Task D - Version Control, Workflows and Deployment	35
4.1 Repository	35
4.2 Version control technique	35
4.3 Version history.....	36
4.4 Workflows (CI/CD)	36
4.5 Evaluation	37
References.....	38

Table of Figures

Figure	Description
Figure 1	Use Case Diagram - Sunrise Dental Clinic System
Figure 2	Class Diagram - Domain Model
Figure 3	Sequence Diagram - Sign In (Authenticate)
Figure 4	Sequence Diagram - Register New Appointment
Figure 5	Sequence Diagram - Calculate the Bill
Figure 6	Three-Tier Architecture and Deployment
Figure 7	Entity-Relationship Diagram - Physical Schema
Figure 8	Sign-in screen
Figure 9	Dashboard showing live clinic figures
Figure 10	Registering a new appointment
Figure 11	Appointment details, found by appointment number
Figure 12	A generated bill and its pricing breakdown
Figure 13	The printable receipt
Figure 14	Revenue report, read from a database view
Figure 15	Audit trail with IP addresses
Figure 16	Notification history
Figure 17	OpenAPI documentation at /swagger-ui.html

Table of Tables

Table	Description
Table 1	Assumptions made during analysis, with reasons
Table 2	Use of <<include>> and <<extend>> stereotypes
Table 3	What belongs in each architectural tier
Table 4	The thirteen design patterns applied
Table 5	Advanced database features and the rules they implement
Table 6	The five management reports and the decisions they support
Table 7	The three levels of testing
Table 8	Test plan
Table 9	Test data, chosen at the boundaries
Table 10	Test results
Table 11	Defects found after the suite was already green
Table 12	Requirements traceability
Table 13	Version control techniques applied
Table 14	Commit history

Task A - System Design with UML Diagrams

Task A (20 marks) - from the assessment brief

“Provide the UML diagrams for the given problem with clear explanations of the design decisions. Derive detailed Use Case diagram, Class diagram and sequence diagram. Whenever necessary document the relevant assumptions you made. (LO D)”

1.1 Understanding the problem

The brief lists four symptoms of running a clinic on paper: double bookings, lost records, long waiting times and billing errors. They are four consequences of one thing - nobody can see, at the moment they need to, what is already true. So the job is not to store the same information in a computer, but to make the conflicting cases **impossible** rather than merely unlikely. That is why a database constraint, not an if statement, does the final work of preventing a double booking (2.4.1), and why an invoice total is calculated twice and compared (2.4.2).

1.2 Assumptions made

#	Assumption	Reason
A1	Only staff use the system; patients do not log in.	The brief says only authorised staff may use it.
A2	Three roles: Receptionist, Dentist, Administrator.	One shared login makes the audit trail meaningless - “who cancelled this?” could never be answered.
A3	One appointment = one visit, one dentist, one treatment.	Keeps the appointment number a genuine unique identifier.
A4	Fixed 30-minute slots, 08:00 to 20:00.	No hours are given. A fixed grid makes “is this slot free?” a question with one correct answer.
A5	Only a completed visit can be billed.	Billing a visit that has not happened is exactly the billing error the clinic wants to escape.
A6	Prices in LKR; VAT 15%.	The clinic is in Colombo. One constant, so a tax change is a one-line change.
A7	Cancelling or missing a visit frees the slot at once.	A slot held by a cancelled visit cannot be sold, recreating the waiting-time problem.

1.3 Use Case diagram

Figure 1 shows three human actors and two system actors. The gateway is an actor because it sits outside the boundary and can fail independently; the scheduler because something must start the daily reminder run when no human is present. The two-column layout is deliberate - the left column holds every use case an actor touches directly, the right only those reached through a stereotype.

1.3.1 Choosing between <<include>> and <<extend>>

These two are often used interchangeably, which empties them of meaning. The rule applied is the standard one (Fowler, 2003): <<include>> happens **every** time; <<extend>> happens **only under a condition**, written on the arrow.

Relationship	Type	Why
Register Appointment → Check Slot Availability	<<include>>	Checked on every booking; skipping it is the double-booking bug the clinic already has.
Register Appointment → Send Confirmation	<<include>>	Every booking confirms; if the gateway is down the attempt is still logged.
Calculate Bill → Apply Pricing Rule	<<include>>	Every bill runs exactly one pricing rule.
Register New Patient → Register Appointment	<<extend>>	[patient not already on file] - a returning patient is looked up.
Reschedule → Display Appointment Details	<<extend>>	[slot must change] - most lookups change nothing.
Print Receipt → Record Payment	<<extend>>	A patient paying by transfer may want no printed receipt.

One omission is deliberate and noted on the diagram: every use case inside the boundary needs a signed-in user, so each strictly includes Sign In, but twelve arrows to one bubble would hide the flow the diagram exists to show.

1.4 Class diagram

Figure 2 is drawn from the classes actually written - the attribute names, types and signatures are the ones in the source. Getters and setters are omitted, because listing seventy would bury the eight methods carrying the business rules. Every attribute is `private` (minus), every business method `public` (plus), and the fields inherited from `BaseEntity` are `protected` (hash). One method is deliberately `private`:

`syncSlotLock()` maintains the field that prevents double booking, and a slot lock disagreeing with the appointment date would silently break that safeguard.

1.4.1 Multiplicity, navigability, aggregation, composition

- **Multiplicity.** A Patient books 0..* Appointments; an Appointment belongs to exactly 1. An Appointment has 0..1 Invoice - zero while scheduled, one once billed - enforced by a unique key on `invoice.appointment_id`, so a visit cannot be billed twice.
- **Navigability.** The arrow to Dentist means an Appointment knows its Dentist but not the reverse; a two-way link would drag a dentist's whole history into memory on every load. Patient-Appointment has no arrowhead because it genuinely is two-way - the patient screen shows their history.
- **Aggregation** (hollow diamond) joins Patient to Appointment: deleting a patient must not silently delete the clinical record of visits they attended.
- **Composition** (filled diamond) joins Invoice to InvoiceLine. A line reading “VAT 15%” means nothing apart from its invoice, so the parts die with the whole - and the foreign key carries ON DELETE CASCADE, so diagram and database agree.

The contrast between the last two is the point. Both read as “has many” and both would be a `List` in Java, but they behave differently on deletion, and the diagram is where that difference is decided.

1.5 Sequence diagrams

Three use cases are shown, because between them they cover the three hardest parts of the system - security, concurrency and business rules.

- **Figure 3 (Sign In)** uses an `alt` fragment because sign-in has two real outcomes, and both end in the same place: an `audit_log` entry with the caller's IP. The same message is shown whether the username exists or not, so the page cannot be used to discover real accounts.
- **Figure 4 (Register Appointment)** is the most important here. The `loop` fragment runs six handlers cheapest-first, so a booking in the past is rejected without touching the database; the `opt` fragment shows the slot being taken in the few milliseconds before the save - a window a paper diary cannot even express.

- **Figure 5 (Calculate Bill)** shows the Template Method fragment: steps 6.1-6.6 always run in that order because the method is `final`, and only 6.2 and 6.3 differ per subclass.

1.6 Critical reflection

The three diagram types answer three questions - **who wants what, what the system is made of, what happens in what order** - and the design holds together only because they agree. One example is worth reporting honestly. The first class diagram had no `slotLock`; the anti-double-booking rule lived only in the sequence diagram, as a check before the save. Drawing that sequence carefully exposed the gap between step 3.4 and step 8 - a gap only something the class owned could close - so `slotLock` was added to the class and the schema as a direct result. The diagrams found a concurrency bug before a line of the service was written.

The main limitation is that the design models one clinic on one server. “One Appointment, one Dentist” would need revisiting if Sunrise opened a second branch, and the fixed 30-minute grid in A4 is a simplification a real orthodontic practice would outgrow.

Figure 1 - Use Case Diagram : Sunrise Dental Clinic System

Appointment & Patient Management | CIS6003 WRIT1 | Task A

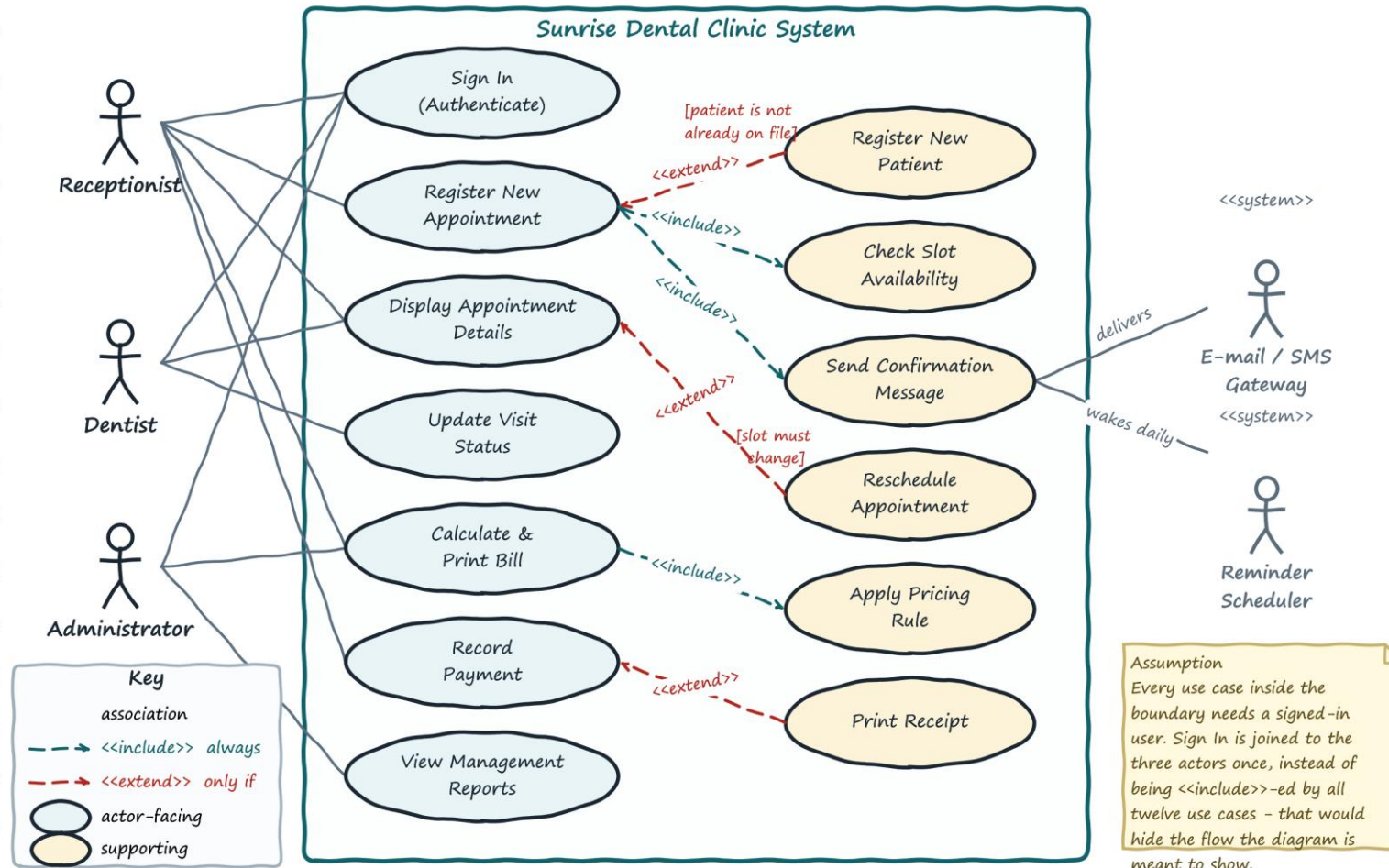


Figure 1 - Use Case Diagram: Sunrise Dental Clinic System. Actors, the system boundary, and the <<include>> / <<extend>> relationships described in section 1.3.

Figure 2 - Class Diagram : Domain Model

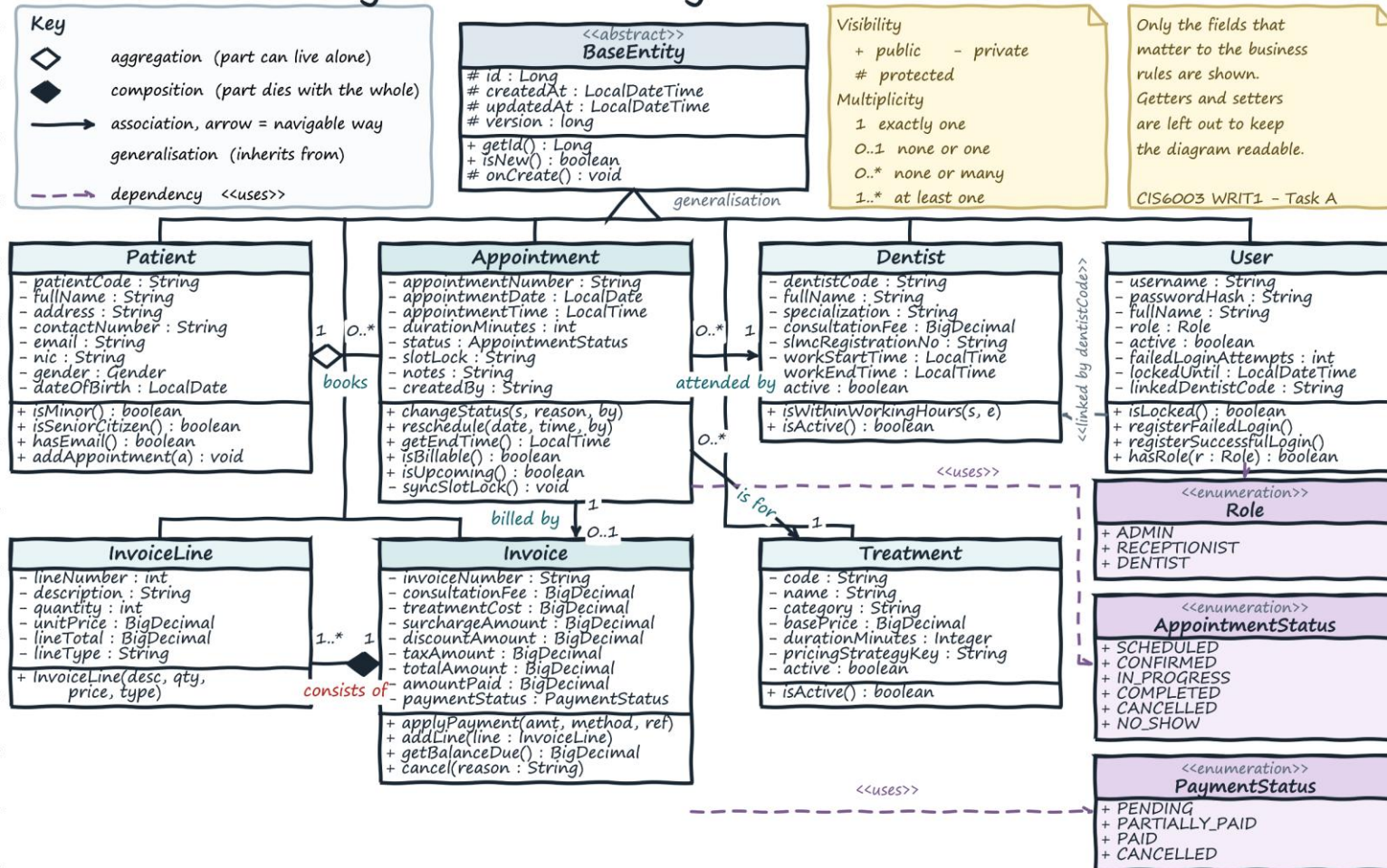
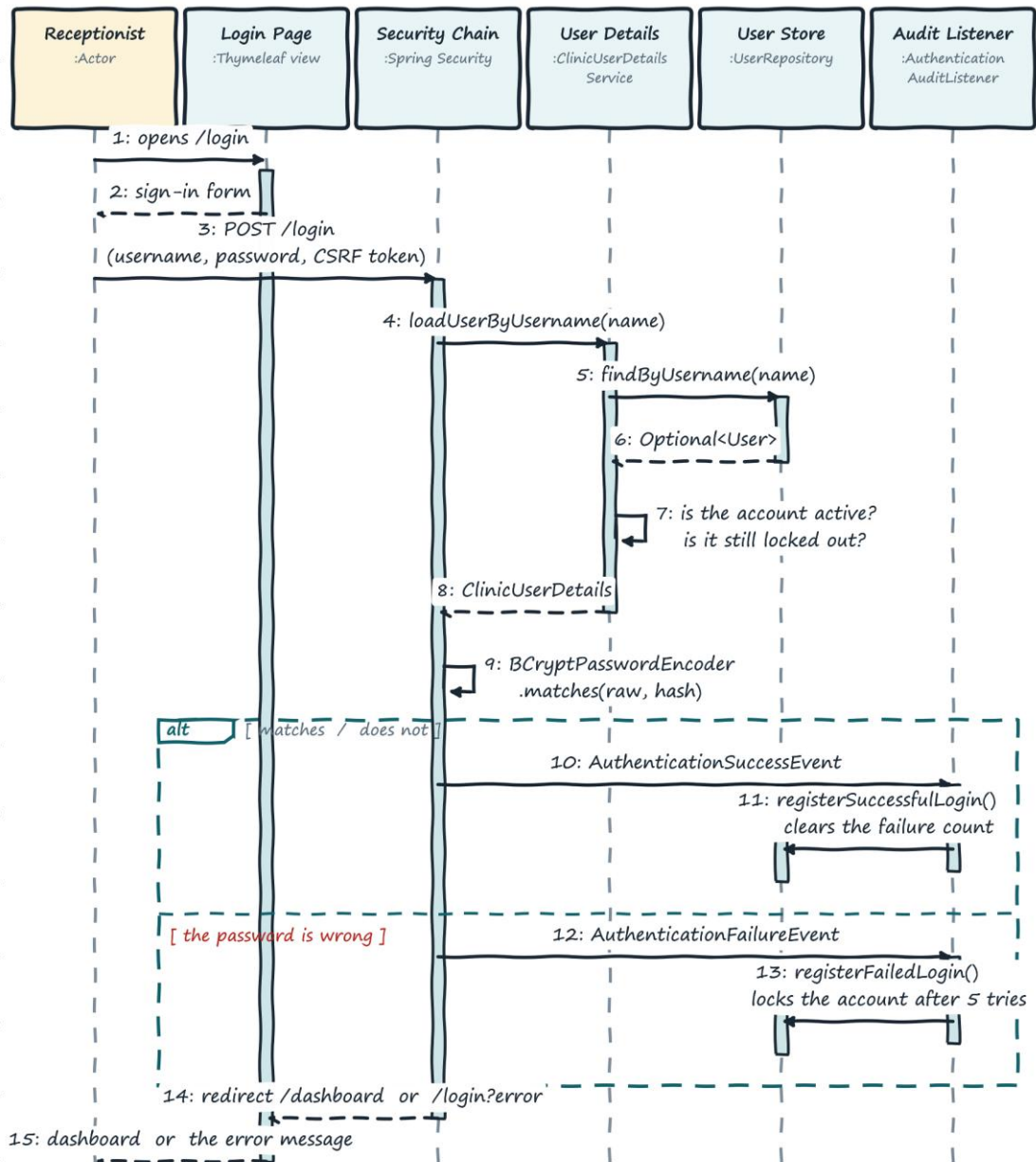


Figure 2 - Class Diagram: the domain model, drawn from the classes that were actually written. Access modifiers, multiplicity, navigability, aggregation and composition are all shown (section 1.4).

Figure 3 - Sequence Diagram : Sign In

Use case UC-01 | CIS6003 WRIT1 | Task A



Task C - security

A password is never stored or compared as plain text. Only its BCrypt hash is held, and the same wrong message is shown whether the username exists or not, so the login page cannot be used to discover valid accounts.

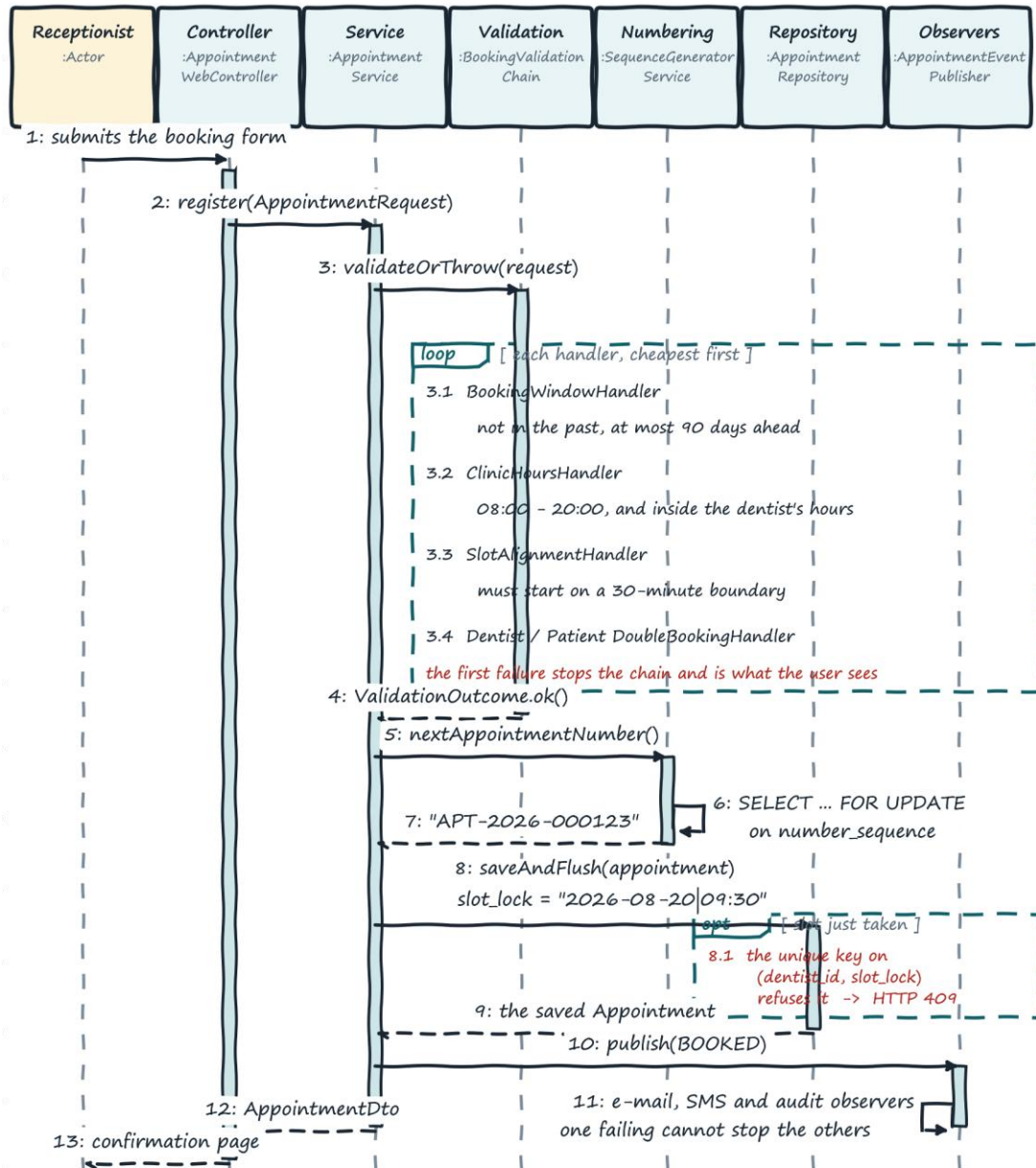
Both branches are audited

Every attempt, successful or not, is written to audit_log with the caller's IP address. The update runs in its own transaction so that two people signing in at once cannot corrupt each other.

Figure 3 - Sequence Diagram: Sign In. The alt fragment shows both outcomes, and both end with an audit entry (section 1.5.1).

Figure 4 - Sequence Diagram : Register Appointment

Use case UC-02 | CIS6003 WRIT1 | Task A

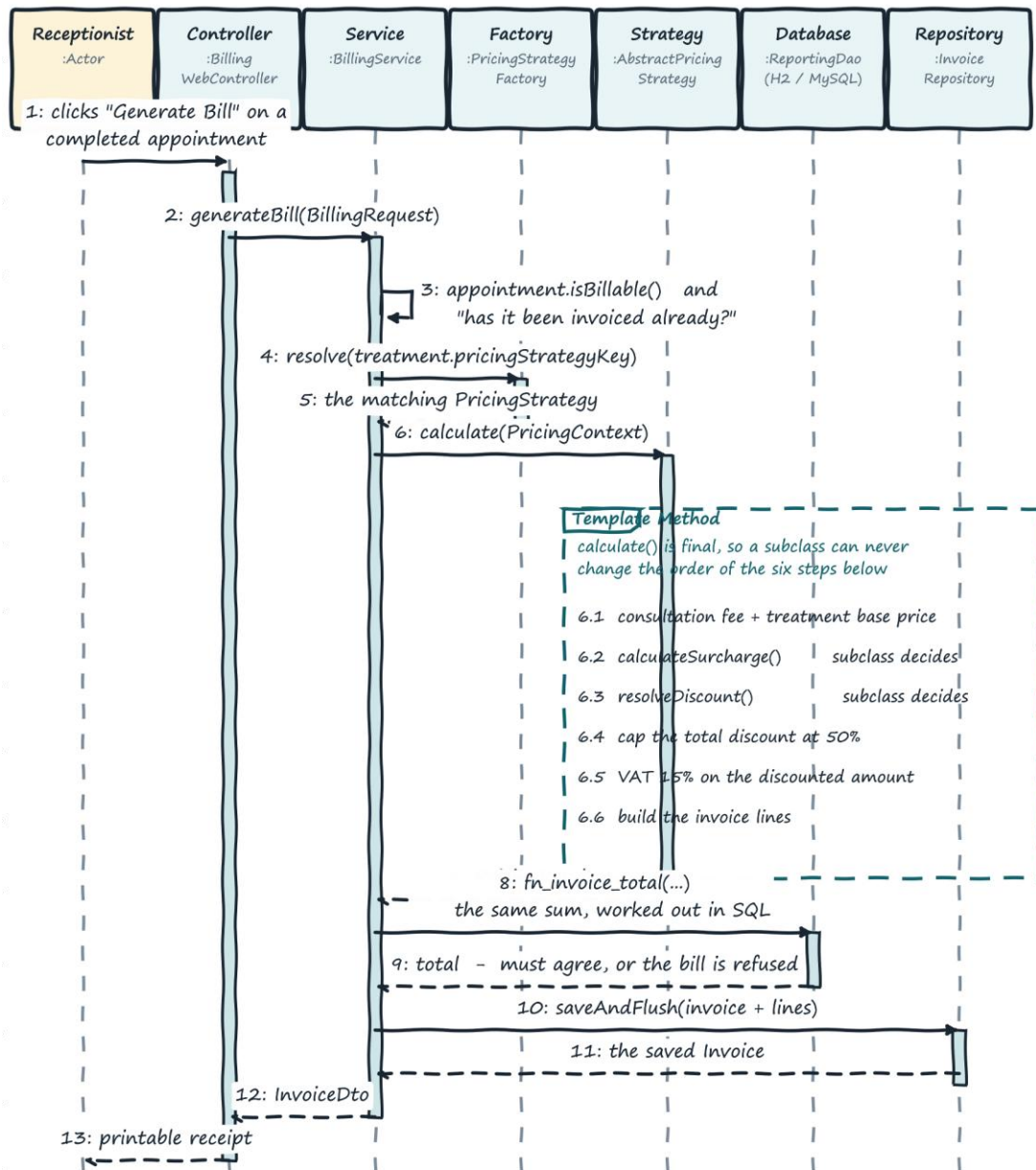


Two independent defences stop the same slot being sold twice. Step 3.4 asks the question; step 8 settles it. The unique key on (dentist_id, slot_lock) cannot be talked round, so even two receptionists pressing Save in the very same instant end up with one booking and one clear "slot unavailable" message. Cancelling a visit sets slot_lock back to NULL, and because NULLs never compare equal in SQL, the freed slot becomes bookable again straight away.

Figure 4 - Sequence Diagram: Register New Appointment. The loop fragment is the validation chain; the opt fragment is the database having the last word on a race (section 1.5.2).

Figure 5 - Sequence Diagram : Calculate the Bill

Use case UC-06 | CIS6003 WRIT1 | Task A



Strategy in action. BillingService never learns which rule ran. STANDARD, SURGICAL, COSMETIC and EMERGENCY each answer steps 6.2 and 6.3 differently - a surgical case adds a theatre surcharge, an emergency adds an out-of-hours loading, a cosmetic case gets no age concession - yet all four are called through the same line of code. Adding a fifth rule means writing one new class, and not one line of this diagram changes. Step 8 is deliberate duplication: the Java total and the SQL total are worked out separately and compared, so a rounding mistake in either one is caught long before a patient is ever handed the bill.

Figure 5 - Sequence Diagram: Calculate the Bill. The Template Method fragment shows the six steps whose order no subclass can change (section 1.5.3).

Task B - Design Patterns and Architecture Development

Task B (40 marks) - from the assessment brief

“Develop an interactive System with set of interfaces to get the necessary user inputs. Make sure to implement proper validation mechanisms in order to restrict invalid entries to the system. Come up with a suitable set of reports, which you think add more value to your system. i. Your program must be a distributed application with web services ii. Appropriate design patterns must be implemented in your system iii. Your program should make use of a proper database to store information. (LO II)”

2.1 Three-tier architecture

Figure 6 shows the architecture. The tiers are not three packages with tidy names; they are separated by a rule applied consistently and open to checking: **a class may only call downwards**. The practical test of a layered design is whether you can delete the top layer and still compile the rest - and here you can, because the desktop client is a different Maven module that does not depend on the server at compile time at all.

Tier	What lives there	What is forbidden there
Presentat ion	Thymeleaf controllers, REST controllers, the Swing client, templates, CSS.	Business rules - a controller that decided a discount would put it out of reach of the desktop client.
Business	Services, ClinicFacade, pricing strategies, the validation chain, observers, security.	SQL and HTTP. It does not know it is reached over the web, which is why the same rules serve both front ends.
Data	Spring Data repositories, ReportingDao, Flyway migrations, views, functions, procedures, triggers.	Decisions. It stores and constrains only.

Three Maven modules - dental-common, dental-server and dental-client - make the build mirror the architecture. Because the client depends only on dental-common, reaching into a server class from the client is impossible: the compiler stops it. Architecture enforced by the build is worth more than architecture enforced by good intentions.

2.2 A genuinely distributed application with web services

This requirement is easy to satisfy in appearance only, by bolting a REST controller onto an ordinary web application. Here it is taken literally: `dental-client` is a Swing application in its own process, with its own JVM, started with the server's address as an argument and able to run on a different machine. It has no compile-time knowledge of any server class, no shared session and no shared object graph - only HTTP and JSON.

The contract is versioned at `/api/v1`, so a future `/api/v2` can change shape without breaking an installed client, and every response uses one envelope - success flag, payload, error code, message - so the client handles failure in one place. It is documented with OpenAPI 3 at `/swagger-ui.html` (Figure 17). One consequence deserves a mention: an API caller must get 401 with a JSON body while a browser user must be redirected to the login page, so Spring Security runs **two separate filter chains**, ordered so `/api/**` matches first. A single chain would have sent HTML login pages to the desktop client.

2.3 Design patterns applied

Pattern	Where	What it buys
Strategy	PricingStrategy + 4 implementations	A new rule is a new class, not an edit to a growing if.
Template Method	AbstractPricingStrategy.calculate()	The order of the billing steps is final.
Factory	PricingStrategyFactory, ReportGeneratorFactory	Callers name what they want; nobody writes new.
Chain of Responsibility	BookingValidationChain (6 handlers)	Each booking rule is independent and testable.
Observer	AppointmentEventPublisher + 3 observers	A failing SMS gateway cannot stop a patient being booked.
Adapter	MessageGateway (console / SMTP / SMS)	A real SMS provider needs no change above the adapter.
Facade	ClinicFacade	Turns a multi-call sequence into one round trip.
Builder	Appointment.Builder	An appointment cannot be half-built.
Singleton	Spring-managed beans	One managed instance per service.
Command	Desktop menu and toolbar actions	Menu bar and button panel cannot drift apart.
DAO / Repository	Spring Data repositories, ReportingDao	Business code never contains SQL.
DTO	Everything in dental-common	Entities never leave the server.
MVC	Thymeleaf controllers,	Request handling is separated from

	templates, domain model	rendering.
--	-------------------------	------------

2.3.1 Strategy and Template Method together

Billing was the natural home for Strategy: four treatment categories have different surcharges and concession rules, and as one method that is a long if/else chain growing with every new category - the classic case the pattern exists to solve (Gamma et al., 1995). What makes it more than a textbook exercise is the combination with Template Method. As pure Strategy each rule would compute its own total, and nothing would stop one applying VAT before the discount instead of after. `calculate()` is therefore final and performs the invariant sequence: fees, surcharge, discount, a 50% cap, VAT, then the invoice lines, with only `calculateSurcharge()` and `resolveDiscount()` left abstract. A subclass decides **how much** but never **in what order** - and since VAT on a discounted amount differs from VAT on an undiscounted one, that distinction is money.

2.3.2 Chain of Responsibility, and Observer

Six conditions must hold before a booking is accepted. As one method that is six nested conditions with a single failure message; as a chain it is six small classes in a fixed order, and the order is a deliberate performance decision logged at start-up: `BOOKING_WINDOW` → `CLINIC_HOURS` → `SLOT_ALIGNMENT` → `DENTIST_AVAILABILITY` → `DENTIST_DOUBLE_BOOKING` → `PATIENT_DOUBLE_BOOKING`. The first three need no database access, so a booking for last Tuesday is rejected without a query, and the chain short-circuits on the first failure so staff are told the most fundamental thing that is wrong.

Observer solves a different problem. Calling the e-mail, SMS and audit code directly from `AppointmentService` means that if the SMS gateway is unreachable the exception propagates and the booking rolls back - the patient loses their appointment because a text message failed. `AppointmentEventPublisher` isolates each observer's failure, logs it and carries on, and every attempt is written to `notification_log` so staff can see the SMS failed and telephone instead.

2.3.3 Critical evaluation - what the patterns cost

- **Indirection.** Following a bill now means opening four files where a beginner would have written one. For four pricing rules that is worth it; for one it would not have been.
- **Debugging is harder.** With Observer a stack trace no longer shows who will react to an event, which is why the registered observers are logged at start-up.
- **A wrong key can fail quietly.** If reference data names a strategy that does not exist, the factory could return null and fail far from the cause. It instead falls back to the standard rule **and logs a warning**.
- **Patterns can be over-applied.** Singleton appears only because Spring manages bean lifecycles; hand-writing one would add global state for no benefit. Naming a pattern is not the same as needing it.

2.4 The database

Figure 7 shows the physical schema: nine tables, five views, two functions, two stored procedures and four triggers. The schema is owned by versioned Flyway scripts and Hibernate runs `ddl-auto=validate`, so the application refuses to start if the entities and the real schema disagree. Letting Hibernate generate the schema would make the database a by-product of the code; this way it is reviewable SQL under version control.

2.4.1 How double booking is actually prevented

The validation chain checks whether a slot is free, but between that check and the insert there is a window of a few milliseconds in which another transaction can commit the same slot. Any solution living only in Java has this window. The fix is a column `slot_lock` and a unique constraint `uk_appointment_slot (dentist_id, slot_lock)`: while an appointment occupies a chair `slot_lock` holds "date|time", and when the visit is cancelled `syncSlotLock()` sets it back to NULL. The trick is that **SQL never treats two NULLs as equal**, so any number of cancelled appointments can share a dentist and slot while two live ones cannot. A database constraint has no race window - and this was verified rather than assumed: twelve genuinely simultaneous requests for one slot gave exactly one success and eleven clean 409 responses, against three fresh databases.

2.4.2 Advanced database features

Feature	Name	Business rule it implements
Function	FN_INVOICE_TOTAL	Recomputes an invoice total in SQL; every bill is checked against it, so a rounding mistake in Java is caught first.
Function	FN_AGEING_BUCKET	Puts a debt in a 0-30 / 31-60 / 61-90 / 90+ day band; the report calls it, so it cannot disagree with an ad-hoc query.
Procedure	SP_SETTLE_INVOICE	Applies a payment and updates the status atomically.
Procedure	SP_DAILY_CLOSING_SUMMARY	Produces the end-of-day figures needed to cash up.
Trigger	trg_appointment_before_insert	Refuses a booking outside the dentist's hours, even from a direct SQL insert (MySQL).
Triggers	trg_appointment_audit_*, trg_invoice_payment_audit	Audit at database level, so a change made outside the application is still recorded.
Views	5 reporting views	Reports read views, so “revenue” is defined once.
Constraints	FK, UNIQUE, CHECK on every table	Discounts <= 50%, payments <= total, status must be permitted.

Two engines are supported from one codebase: H2 in file mode by default, so the application runs with nothing to install, and MySQL 8 through a Spring profile. The MySQL path was tested end to end against a real MariaDB 10.4 server - all four migrations, both functions, both procedures and every trigger exercised with real data, not merely read through.

2.5 Interfaces and validation

Two complete front ends were built. The browser application (Figures 8 to 17) covers all six required functions plus patient, dentist, treatment, report and administration screens, using a hand-written stylesheet with no CDN dependency so the clinic stays usable if its internet drops. The desktop client is menu-driven, as the brief asks, with the Command pattern behind the menu bar and the button panel.

Validation is applied in four places, on the principle that each layer should refuse what it can prove is wrong: HTML5 required, type and pattern attributes in the **browser**; Bean Validation annotations on the **DTOs**, so the desktop client enforces identical rules with no duplicated code; the six-handler chain in the **business tier**; and CHECK, UNIQUE and FOREIGN KEY constraints plus the MySQL trigger in the

database, which hold even against a direct SQL statement. Messages are written for a receptionist, not a developer: instead of a constraint violation the user is told which slot is taken and what to do about it.

2.6 Reports that add value

Report	Question it answers	Decision it supports
Daily Schedule	Who is coming in today, to which dentist?	Staffing the desk; preparing surgeries.
Revenue	What did the clinic take, by day and treatment?	Whether a treatment is worth continuing to offer.
Outstanding Invoices	Who owes money, and for how long?	Which debts to chase first.
Dentist Workload	How busy is each dentist?	Rebalancing the rota; whether to recruit.
Treatment Popularity	Which treatments are most in demand?	Stock ordering; continuing-education choices.

Each is justified by the decision it supports rather than the data it contains. All five share one envelope class, which is why one web page and one desktop window display any of them, and why a sixth report would need no user-interface work at all.

2.7 Sessions and cookies

Three uses are implemented, each solving a real problem rather than demonstrating a mechanism. The **HTTP session** carries the signed-in user and a “recently viewed” trail, so a receptionist interrupted mid-task can return to where they were. An opt-in **remember-me cookie** saves the front-desk machine signing in every morning, and a **preference cookie** stores the chosen table density. A CSRF token is required on every form, so a stolen session cookie alone cannot submit one.

Figure 6 - Three-Tier Architecture and Deployment

CIS6003 WRIT1 | Task B

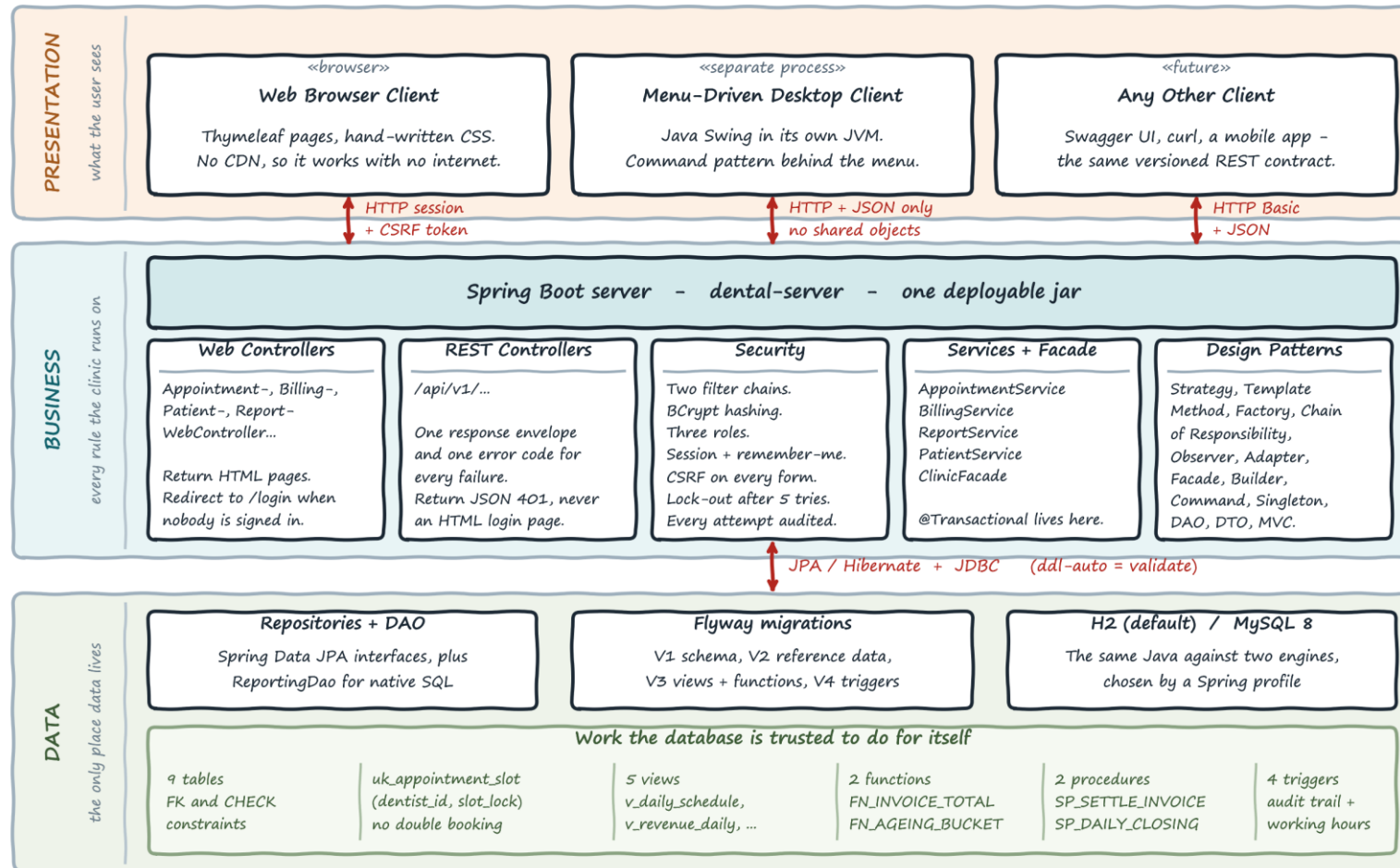


Figure 6 - Three-tier architecture and deployment. The desktop client reaches the server over HTTP and JSON only, which is what makes the application genuinely distributed (section 2.2).

Figure 7 - Entity-Relationship Diagram : Physical Schema

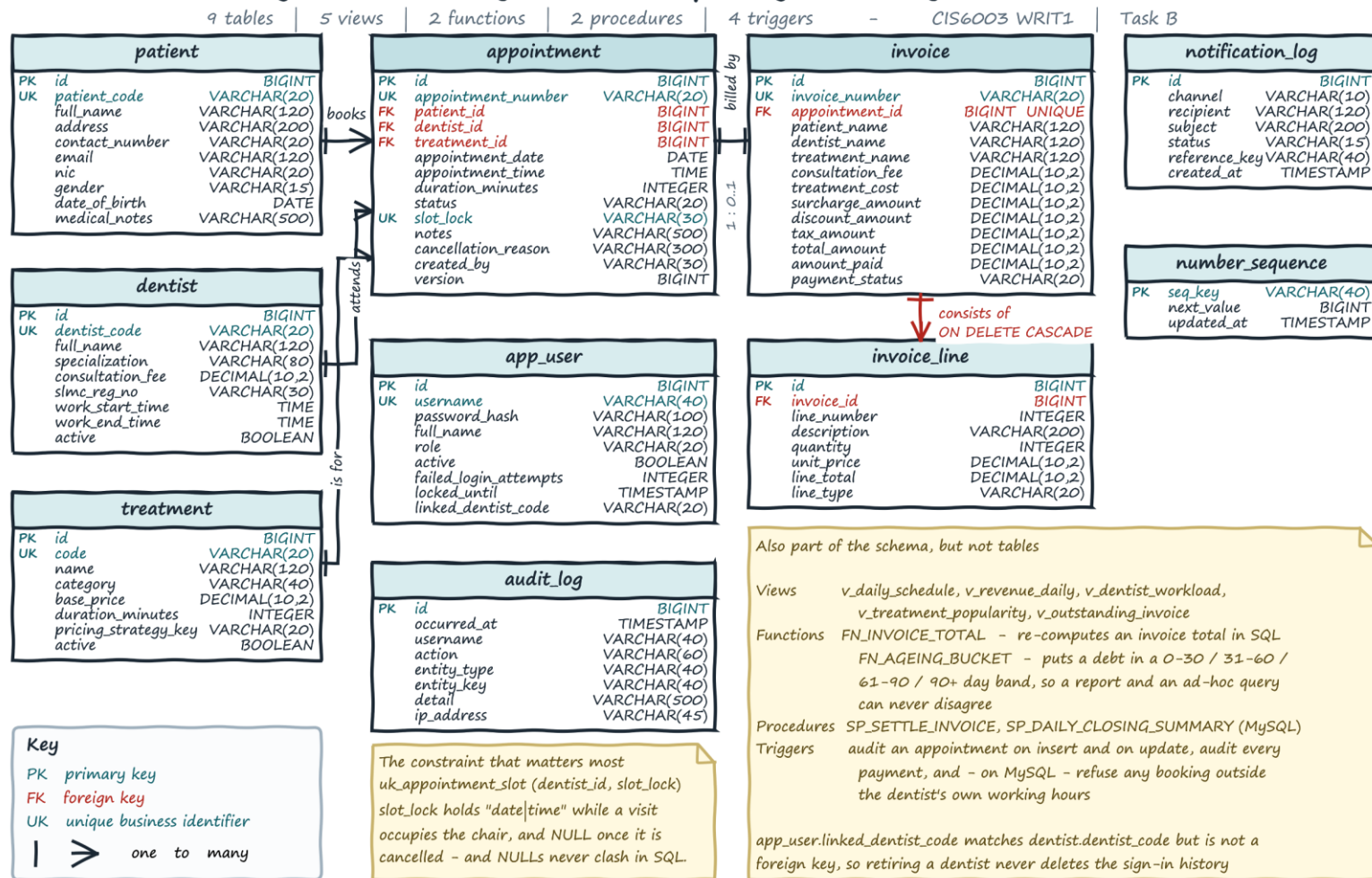


Figure 7 - Entity-Relationship diagram of the physical schema. Note uk_appointment_slot on (dentist_id, slot_lock), which is what actually prevents double booking (section 2.4.1).

2.8 The system in use

The screenshots below are of the running application with seeded clinic data. They are included because the marking criteria ask for documentation with screenshots and clear explanations, and because a description of an interface is no substitute for seeing it.

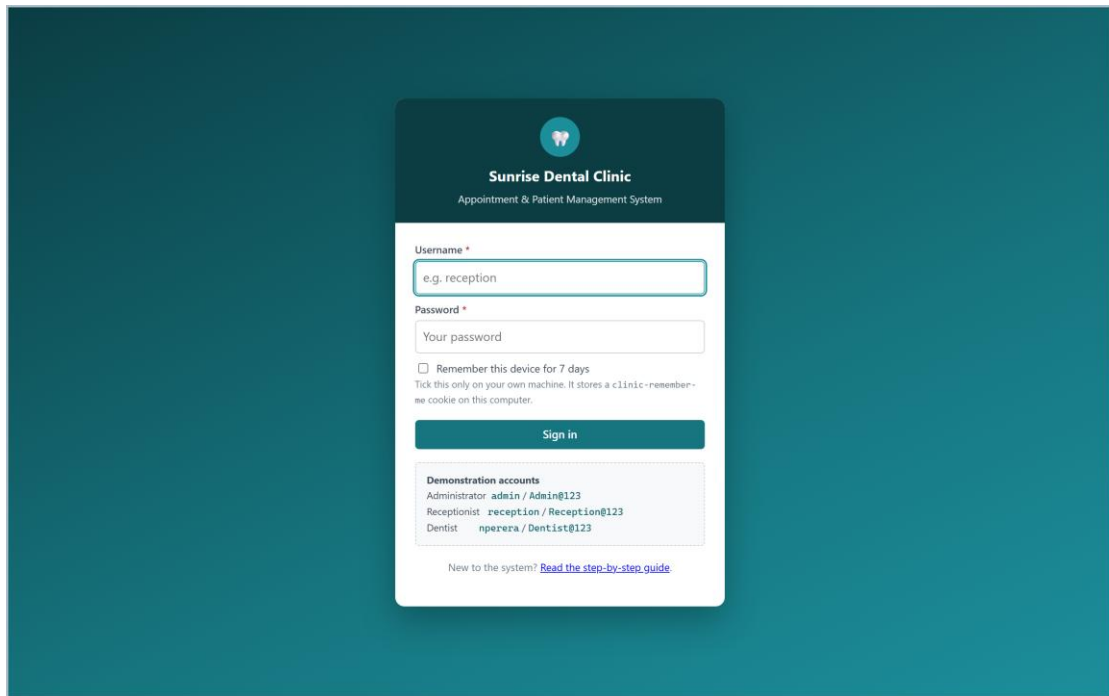


Figure 8 - The sign-in screen. The same message is shown for an unknown username and a wrong password, so the page cannot be used to discover which accounts exist.

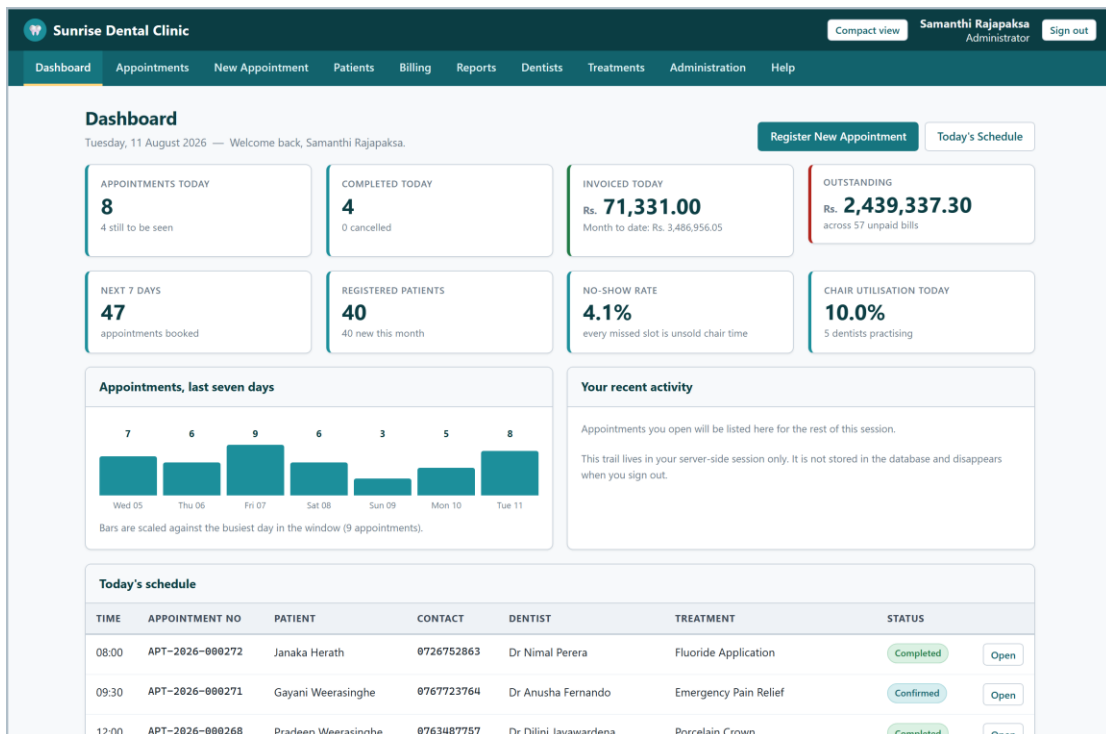


Figure 9 - The dashboard, showing live figures: today's appointments, outstanding invoices and revenue to date.

Sunrise Dental Clinic | Compact view | Samantha Rajapaksa Administrator | Sign out

Register New Appointment
The appointment number is issued automatically once the booking is saved. [Back to appointments](#)

Patient details

Returning patient? Enter their patient number (PAT-000042) and the record on file is used. [Search for a patient](#).
New patient? Leave the patient number blank and complete the fields below — they will be registered automatically.

Existing patient number: PAT-000042 (leave blank for a new pat) | Patient name *: e.g. Kamala Perera

Address *: No. 45, Galle Road, Colombo 03

Contact number *: 0771234567 | E-mail address: optional | NIC: 199012345678 or 901234567V | Date of birth: mm/dd/yyyy

Gender: Not stated

Appointment details

Dentist *: -- select a dentist -- | Treatment type *: -- select a treatment -- | Appointment date *: mm/dd/yyyy | Appointment time *: --:--:--

Notes: Anything the dentist should know before the visit

Not sure which slots are free? Choose a dentist and a date, then use Check availability below — the system shows only slots that are genuinely open.

Figure 10 - Registering a new appointment. The dentist, treatment and slot are chosen from reference data, so an invalid combination cannot be typed in.

Sunrise Dental Clinic

Compact view
Samanthi Rajapaksa
Administrator
Sign out

Dashboard
Appointments
New Appointment
Patients
Billing
Reports
Dentists
Treatments
Administration
Help

APT-2026-000274

Completed
Registered by reception on 11 Aug 2026 21:34

Back to list
Patient record

Patient

Patient number: PAT-000008
Name: Dilshan Jayawardena
Address: No. 36, Duplication Road, Colombo 03
Contact number: 0775262604
E-mail: Not supplied

Appointment

Date: Tuesday, 11 August 2026
Time: 12:00 – 12:30 (30 minutes)
Dentist: Dr Kasun Silva Oral & Maxillofacial Surgery
Treatment: General Consultation (Rs. 1500.00)
Notes: None

What happened?

This appointment is **Completed** and can no longer be changed. If this is wrong, ask an administrator — the correction is recorded against their name.

Bill

Bill INV-2026-000239 was issued on 11 Aug 2026 for **Rs. 5,310.00** — Paid

Open bill
Print receipt

This patient's visit history (16 visits)

Appointment No	Date	Dentist	Treatment	Status
----------------	------	---------	-----------	--------

Figure 11 - The appointment detail screen, reached by searching for the appointment number as the brief requires.

Sunrise Dental Clinic

Compact view
Samanthi Rajapaksa
Administrator
Sign out

Dashboard
Appointments
New Appointment
Patients
Billing
Reports
Dentists
Treatments
Administration
Help

INV-2026-000239

Paid
Issued by reception on 11 Aug 2026 21:34

Print receipt
Download (.txt)
Appointment

Bill to

Patient: Dilshan Jayawardena
Address: No. 36, Duplication Road, Colombo 03
Contact: 0775262604
Appointment: APT-2026-000274
Visit date: 11 Aug 2026
Dentist: Dr Kasun Silva
Treatment: General Consultation
Pricing rule: STANDARD

These details are the copy taken when the bill was issued. If the patient later changes address, this historic bill deliberately does not change with it.

Amount

DESCRIPTION	AMOUNT
Consultation fee	3,000.00
General Consultation	1,500.00
VAT @ 18%	810.00
TOTAL PAYABLE	Rs. 5,310.00
PAID SO FAR	5,310.00
BALANCE DUE	Rs. 0.00

Sunrise Dental Clinic — No. 172, Galle Road, Colombo 03 | Tel: +94 11 2 573 100
CIS6003 Advanced Programming — WRIT1 | [REST API documentation](#) | [Help](#)

Figure 12 - A generated bill, showing the full pricing breakdown: consultation fee, treatment cost, surcharge, discount, VAT and total.

Sunrise Dental Clinic

Compact view
Samanthi Rajapaksa
Administrator
Sign out

Dashboard
Appointments
New Appointment
Patients
Billing
Reports
Dentists
Treatments
Administration
Help

Receipt

Bill INV-2026-000239 for Dilshan Jayawardena

Print this receipt
Download as text
Back to bill

=====
SUNRISE DENTAL CLINIC
No. 172, Galle Road
Colombo 03, Sri Lanka
Tel: +94 11 2 573 100
Reg. No: SLMC/DC/2011/0842
=====

=====
PATIENT BILL / RECEIPT
=====
Bill No : INV-2026-000239
Appointment No : APT-2026-000274
Date : 11 Aug 2026 21:34
Issued By : reception
=====
Patient : Dilshan Jayawardena
Patient No : PAT-000008
Address : No. 36, Duplication Road, Colo
: mbo 03
Contact : 0775262604
=====
Dentist : Dr Masun Silva
Treatment : General Consultation
Visit Date : 11 Aug 2026 12:00
=====
DESCRIPTION AMOUNT (LKR)
=====
Consultation fee 3,000.00
General Consultation 1,500.00
VAT @ 18% 810.00
=====
Sub-total 4,500.00
Taxable amount 4,500.00
VAT @ 18% 810.00
=====
TOTAL PAYABLE 5,310.00
=====
Amount paid 5,310.00
Balance due 0.00
Status : Paid
Paid by : Cash
Pricing rule : STANDARD
=====
Thank you for choosing
Sunrise Dental Clinic
=====

Figure 13 - The printable receipt. The browser and the desktop client both print this same server-rendered page, so the two can never disagree.

Sunrise Dental Clinic

Compact view
Samanthi Rajapaksa
Administrator
Sign out

Dashboard
Appointments
New Appointment
Patients
Billing
Reports
Dentists
Treatments
Administration
Help

Revenue Analysis

Invoiced versus collected income per day, with discounts and VAT separated, so that pricing decisions and collection performance can be judged apart.

Print
All reports

Report period

From To
mm/dd/yyyy mm/dd/yyyy
Generate

Dates default to the current month. The maximum range is one year. The debtor ageing report ignores the dates on purpose — a four-month-old unpaid bill is exactly what it exists to surface.

Summary

Period 01 Aug 2026 to 11 Aug 2026 | generated 11 Aug 2026 21:35 by admin | 11 rows

TRADING DAYS WITH INCOME
11

INVOICES ISSUED
54

GROSS CHARGES
Rs. 2,995,950.00

DISCOUNTS GIVEN
Rs. 40,902.50

VAT COLLECTED
Rs. 531,908.55

TOTAL INVOICED
Rs. 3,486,956.05

TOTAL COLLECTED
Rs. 3,042,069.50

STILL OUTSTANDING
Rs. 444,886.55

COLLECTION RATE
87.2%

AVERAGE BILL
Rs. 64,573.26

AVERAGE PER DAY
Rs. 316,996.00

Revenue Analysis

DATE	INVOICES	GROSS	DISCOUNT	VAT	NET INVOICED	COLLECTED	OUTSTANDING
01 Aug 2026	6	123,550.00	1,052.50	22,049.55	144,547.05	134,402.00	10,145.05
02 Aug 2026	2	324,200.00	0.00	58,356.00	382,556.00	382,556.00	0.00

Figure 14 - The revenue report, read from the v_revenue_daily database view.

Sunrise Dental Clinic							Compact view	Samanthi Rajapaksa Administrator	Sign out
Dashboard	Appointments	New Appointment	Patients	Billing	Reports	Dentists	Treatments	Administration	Help

Audit Trail

Who did what, and when. Append-only — entries are never edited or deleted.

Notification history
Staff accounts
System diagnostics

Filter

Username

e.g. reception

Action contains

LOGIN, APPOINTMENT, INVOICE, ...

Filter

Clear

Entries (405)

WHEN	USER	ACTION	ENTITY	DETAILS	SOURCE	IP
11 Aug 21:35:06	admin	LOGIN_SUCCESS	USER admin	Signed in successfully	Application	0:0:0:0:0:1
11 Aug 21:34:19	reception	APPOINTMENT_CREATED	APPOINTMENT APT-2026-000404	Booked for 2026-09-01 18:00, status CONFIRMED	Database	-
11 Aug 21:34:19	reception	APPOINTMENT_CREATED	APPOINTMENT APT-2026-000403	Booked for 2026-09-01 10:00, status CONFIRMED	Database	-
11 Aug 21:34:19	reception	APPOINTMENT_CREATED	APPOINTMENT APT-2026-000402	Booked for 2026-09-01 15:00, status CONFIRMED	Database	-
11 Aug 21:34:19	reception	APPOINTMENT_CREATED	APPOINTMENT APT-2026-000401	Booked for 2026-08-31 15:00, status CONFIRMED	Database	-
11 Aug 21:34:19	reception	APPOINTMENT_CREATED	APPOINTMENT APT-2026-000400	Booked for 2026-08-31 15:00, status SCHEDULED	Database	-
11 Aug 21:34:19	reception	APPOINTMENT_CREATED	APPOINTMENT APT-2026-000399	Booked for 2026-08-31 08:30, status CONFIRMED	Database	-
11 Aug 21:34:19	reception	APPOINTMENT_CREATED	APPOINTMENT APT-2026-000398	Booked for 2026-08-31 14:00, status CONFIRMED	Database	-
11 Aug 21:34:19	reception	APPOINTMENT_CREATED	APPOINTMENT APT-2026-000397	Booked for 2026-08-31 13:30, status CONFIRMED	Database	-
11 Aug 21:34:19	reception	APPOINTMENT_CREATED	APPOINTMENT APT-2026-000396	Booked for 2026-08-30 14:00, status SCHEDULED	Database	-

Figure 15 - The audit trail, showing every sign-in and every change with the caller's IP address.

Sunrise Dental Clinic							Compact view	Samanthi Rajapaksa Administrator	Sign out
Dashboard	Appointments	New Appointment	Patients	Billing	Reports	Dentists	Treatments	Administration	Help

Notification History

Every e-mail and SMS the system attempted to send.

Send tomorrow's reminders now
Audit trail

Development mode

E-mail is delivered by the console gateway: messages are recorded here and written to the application log, but not transmitted. SMS is simulated - numbers are normalised to E.164 and validated exactly as a real aggregator would, so failures are genuine. Set `clinic.notifications.smtp-enabled=true` to send real e-mail.

Messages (0)

WHEN	CHANNEL	RECIPIENT	REFERENCE	SUBJECT / MESSAGE	STATUS
No notifications have been sent yet. Register an appointment, or use "Send tomorrow's reminders now" above.					

Sunrise Dental Clinic

— No. 172, Galle Road, Colombo 03 | Tel: +94 11 2 573 100

CIS6003 Advanced Programming — WRIT1

| [REST API documentation](#) | [Help](#)

Figure 16 - The notification history, recording every alert attempt, successful or failed.

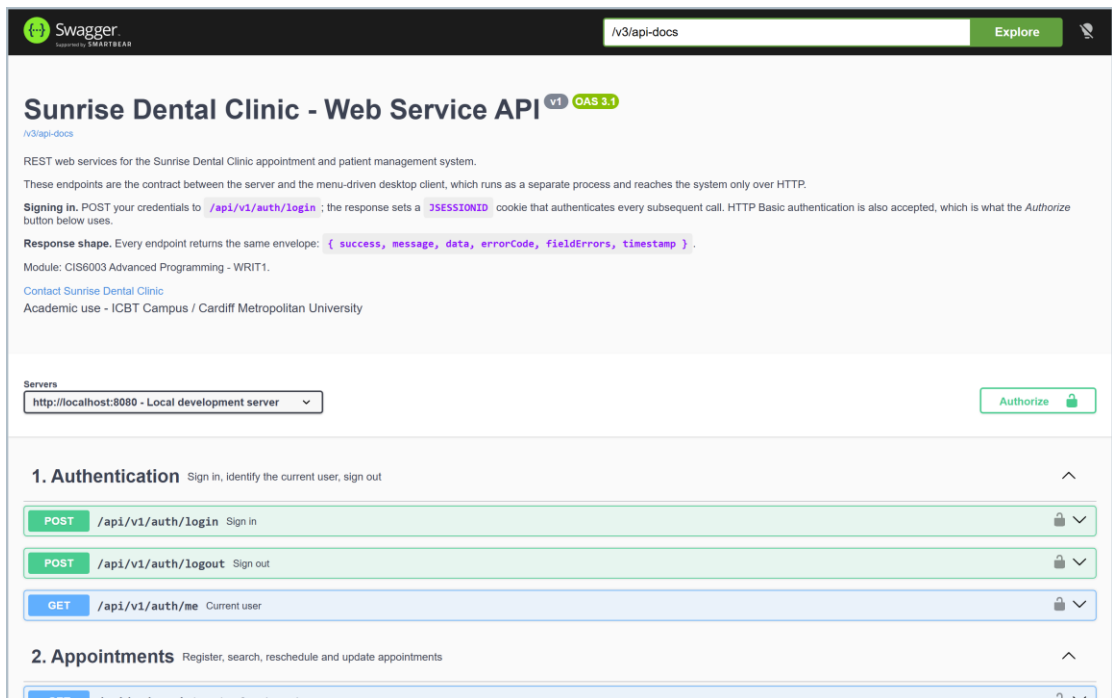


Figure 17 - The OpenAPI documentation at /swagger-ui.html, which the desktop client and any future client are written against.

Task C - Test Plan and Test-Driven Development

Task C (20 marks) - from the assessment brief

“Document the test plan and explain how you used test-driven development in this scenario and do a test automation to achieve that. This includes test rationale, test plan, test data and proper application of the test plan. (LO II)”

3.1 Test rationale

The clinic's problems are all failures of correctness under pressure: two bookings for one slot, a bill that adds up wrongly, a record that cannot be found. The suite therefore has to prove the rules hold when they are attacked, not merely that the happy path works. Three levels were used, so each failure is diagnosed as cheaply as possible.

Level	What it proves	How it runs
Unit (226)	One class behaves correctly in isolation.	JUnit 5 with Mockito - no Spring context, no database, so a failure points at one class.
Integration (97)	The parts work together against the real schema.	@SpringBootTest + MockMvc on the real Flyway schema, in-memory H2.
Concurrency / manual	The rules hold under a genuine race.	A multi-threaded client, plus scripted page and endpoint checks.

Maven runs the levels in separate phases - Surefire for units, Failsafe for integration - so a broken unit test stops the build before the slow tests begin.

3.2 How test-driven development was used

TDD was applied where it genuinely helps and not where it would have been theatre. The billing rules, the appointment lifecycle and the validation chain were written test-first in the red-green-refactor cycle (Beck, 2003), because they have clear inputs, clear outputs and rules statable in a sentence. Before any code existed, a test asserted that a CANCELLED appointment cannot become COMPLETED; writing it forced a decision that had been vague until then, and the resulting enum owns the rule so no service can bypass it. TDD was **not** used for the Thymeleaf templates or the Swing windows - a failing test for the position of a form field would lock in a layout before anyone had

looked at it - so those were built, reviewed by eye, then covered by integration tests asserting on behaviour rather than markup.

One case shows the cycle earning its keep. A test asserted that rescheduling must not report a clash with itself. It failed, as expected; the interesting part was **why** - the double-booking query excluded nothing, so an appointment was its own conflict. That is a bug a manual tester would have found only after a receptionist reported it.

3.3 Test plan

ID	Area	What is tested	Type	Result
TP-01	Authentication	Valid sign-in, wrong password, unknown user, locked account, CSRF, role enforcement, sign-out	Integration	Pass
TP-02	Register appointment	Valid booking, six validation rules, duplicate slot, unknown patient / dentist / treatment	Unit + Int.	Pass
TP-03	Appointment lifecycle	Every legal status transition, and every illegal one refused	Unit	Pass
TP-04	Display details	Search by appointment number; unknown number; complete data shown	Integration	Pass
TP-05	Billing	Four pricing rules, VAT after discount, 50% cap, senior and child concessions, rounding	Unit	Pass
TP-06	Billing integrity	Java total equals FN_INVOICE_TOTAL; a visit is billable once only	Integration	Pass
TP-07	Payments	Full, partial and over-payment; status transitions; receipt	Unit + Int.	Pass
TP-08	Database features	Views return rows, functions compute correctly, triggers write audit rows, CHECK constraints refuse bad data	Integration	Pass
TP-09	Reports	All five reports; empty and invalid ranges; unknown report name	Integration	Pass
TP-10	Web services	Every /api/v1 endpoint, the envelope, error codes, JSON 401 not an HTML redirect	Integration	Pass
TP-11	Web interface	Every page renders; navigation works; role-based menus	Integration	Pass
TP-12	Concurrency	12 simultaneous bookings for one slot; sequence allocation under load	Automated	Pass
TP-13	Desktop client	Connects over HTTP only; all menu commands; server-rendered receipt	Manual probe	Pass

3.4 Test data

Data was devised to cover boundaries rather than typical values, because a bug almost never lives in the middle of a range.

Category	Examples used
----------	---------------

Valid, ordinary	A 34-year-old patient; a routine filling at 10:00 next Tuesday; paid in full by card.
Boundary - age	Exactly 18 (not a minor by one day), exactly 60 (senior concession), 17 and 61 either side.
Boundary - time	08:00 (first legal slot), 20:00 (first illegal), 09:29 and 09:30 either side of the grid.
Boundary - money	A discount of exactly 50% (allowed) and 50.01% (refused); a payment equal to the total, and one cent more.
Boundary - dates	Today; yesterday (refused); 90 days ahead (allowed) and 91 (refused).
Invalid	Blank name, 9-digit telephone number, negative price, unknown treatment code, forbidden status transition.
Malicious	A form posted with no CSRF token; a RECEPTIONIST requesting an admin page; an API call with no credentials.
Concurrent	Twelve threads released simultaneously against one dentist, one date, one time.

Data comes from a `TestDataFactory`, so a change to an entity is a change in one place. Identifiers are generated from a sequence, which found a real defect early: one version produced appointment numbers longer than the `VARCHAR(20)` column, failing for exactly the reason a busy year would eventually have failed in production.

3.5 Test automation and results

The whole suite is automated and runs from one command, `mvn verify`. Nothing needs installing and no database needs starting, because integration tests use an in-memory H2 instance that Flyway migrates on the way up; JaCoCo produces coverage in the same run. The same command runs in CI on every push, on both JDK 17 and JDK 21, and the workflow then packages both jars and **boots the packaged server** to prove it starts - a suite can pass while an application fails to start.

Measure	Result
Unit tests	226 - all passing; 0 failures, 0 errors, 0 skipped
Integration tests	97 - all passing; 0 failures, 0 errors, 0 skipped
Total	323 automated tests, all passing
Test classes	31 (26 unit, 5 integration)
Server coverage (JaCoCo)	73.7% lines, 76.4% methods, 96.6% classes (115 of 119)
Build	<code>mvn verify</code> green on JDK 17 and JDK 21

Coverage deserves an honest note. `dental-common` reports far lower coverage because JaCoCo reports per module: its DTOs are exercised heavily by the server's integration tests, but that execution is credited to the server's report, and what remains uncovered

is overwhelmingly getters and setters. Writing tests for generated accessors would raise the percentage without raising the quality.

3.6 What the testing actually found

A test report saying everything passed first time is not one anybody should believe. Three genuine defects were found **after** the suite was green, all by the concurrency test in section 2.4.1. Two shared one root cause worth stating plainly: **a failure that surfaces at transaction commit, inside Spring's proxy, cannot be caught by a try/catch inside the same '@Transactional' method.**

Defect	Symptom	Fix
Login bookkeeping race	Two requests authenticating as the same account in the same instant raced to update one versioned User row. The loser's locking failure escaped as a raw HTTP 500, and its audit entry was lost.	Moved the update into its own transactional bean so the listener's try/catch can see the failure; the audit entry is now written unconditionally.
First sequence allocation	SELECT ... FOR UPDATE cannot lock a row that does not exist yet, so the first appointment number of a year could be claimed by several bookings at once, and every loser's booking was abandoned.	A one-time retry inside a genuinely new transaction (SequenceAllocator).
CI action reference	trivy-action@0.24.0 was referenced, but that action's tags carry a v prefix, so the job would have failed on its first run.	Corrected to @v0.24.0; every action reference was then verified against the GitHub API.

The lesson generalises: a sequential test proves a rule is **written**, but only a concurrent test proves it **holds**. Both would have reached production and been very hard to reproduce, because they appear only when two people act in the same instant - precisely what a busy front desk creates every day.

3.7 Traceability

Requirement	Design	Implementation	Verified by
1. User authentication	Fig. 1, 3	Spring Security, two filter chains, BCrypt	TP-01, SecurityIT
2. Register appointment	Fig. 1, 2, 4	AppointmentService.register() + validation chain	TP-02, AppointmentServiceTest, AppointmentApiIT
3. Display details	Fig. 1, 2	requireByNumber(), web and API	TP-04, AppointmentApiIT
4. Calculate and print bill	Fig. 1, 5	BillingService + four pricing strategies	TP-05, TP-06, PricingStrategyTest, BillingApiIT
5. Help section	Fig. 1	HelpService, /help pages	TP-11, ReportAndWebUiIT
6. Exit system	Fig. 1	POST-only sign-out; client exit command	TP-01, SecurityIT

Distributed + web services	Fig. 6	/api/v1, separate Swing process	TP-10, TP-13
Design patterns	Fig. 2, 5	13 patterns (section 2.3)	TP-05, PricingStrategyFactoryTest, BookingValidationChainTest
Proper database	Fig. 7	9 tables, 5 views, 2 functions, 2 procedures, 4 triggers	TP-08, DatabaseFeaturesIT, ClinicFunctionsTest
Validation	Fig. 4	Four layers (section 2.5)	TP-02, ValidationConstraintsTest
Reports	Fig. 6	Five reports over database views	TP-09, ReportAndWebUiIT

3.8 Evaluation and lessons learned

Every business rule in the brief is covered, the rules involving money are covered at their boundaries, and the suite runs unattended in under two minutes. Three lessons stand out. **The cheapest bug is the one a test finds before the code exists** - the lifecycle rules cost almost nothing because they were specified as tests first. **Coverage is a smell detector, not a score**: the concurrency defects were in code already covered. And **tests should mirror how the software will actually be used** - a front desk has two people booking at once, and once the harness reflected that it immediately found two faults every sequential test had passed over. If extended, the next step would be property-based testing of the pricing rules.

Task D - Version Control, Workflows and Deployment

Task D (20 marks) - from the assessment brief

“Create your own Git/ GitHub repository which is public to access and upload /deploy the changes of the software project you have developed in it. Share the report link within the documentation. Update it with several versions where modifications are applied each day... Version control techniques you have used throughout the development should be highlighted and documented properly. Demonstrate workflows deployed with the Git repository. (LO III)”

4.1 Repository

Repository URL: [https://github.com/\[your-github-username\]/sunrise-dental-clinic](https://github.com/[your-github-username]/sunrise-dental-clinic)

The repository is public, so it opens without a GitHub account. It holds the complete Maven project, the migration scripts, the test suite, both CI workflows, this documentation and the editable draw.io source for every figure.

4.2 Version control technique

The history was built deliberately: nineteen commits in fifteen numbered increments, each a complete working step, so the repository shows how the application was constructed rather than appearing fully formed in one commit. Any commit can be checked out and built.

Technique	How it was applied	Why
Conventional Commits	feat(pricing):, fix(security):, test:, ci:, docs:	The subject says what changed and where, so the log is searchable.
Meaningful increments	One commit per layer or feature	A reviewer can follow the reasoning; a single “final submission” commit hides it.
Branching	main holds released work; develop integrates	A simplified Git Flow (Driessen, 2010); a full feature-branch model would be ceremony for one developer.
Semantic versioning	Annotated tags v1.0.0 and v1.0.1	The number states whether an upgrade is safe.
Tag-driven release	Pushing a v*.*.* tag triggers the release workflow	A deployed version always traces back to one immutable commit.
.gitignore discipline	Build output, IDE files, the H2 data directory and credentials are excluded	The repository is public; patient data must never reach it.

Changelog	CHANGELOG.md in Keep a Changelog format	Explains why each release exists, which a commit log alone does not.
-----------	---	---

4.3 Version history

#	Commit	What it added
1	chore: scaffold multi-module Maven project	Three-module reactor, Java 17, pinned plugin versions.
2	feat(common): shared service contract	DTOs, enums and validation constraints shared by both sides.
3	feat(data): build the data tier	Flyway schema for H2 and MySQL, entities, views, functions, triggers, slot_lock.
4	feat(pricing): billing rules	Strategy, Template Method, Factory - four pricing rules.
5	feat(validation): booking rules	Chain of Responsibility - six independent handlers.
6	feat(notifications): notification pipeline	Observer and Adapter - e-mail, SMS and audit observers.
7	feat(reports): five management reports	Template Method and Factory over the database views.
8	feat(services): services and the Facade	Services, ClinicFacade, sequence generation under a lock.
9	feat(security): authentication and sessions	Two filter chains, BCrypt, roles, remember-me, lock-out.
10	feat(api): publish the REST web services	Versioned /api/v1, one response envelope, OpenAPI 3.
11	feat(web): Thymeleaf web user interface	All six required functions plus admin and reports.
12	feat(client): menu-driven desktop client	Swing client in a separate process, Command pattern.
13	test: unit and integration test suite	323 automated tests; JaCoCo wired into verify.
14	ci: add CI and release pipelines	Build matrix, tests, packaging, smoke test, release on tag.
15	docs: README, changelog, version-control notes	Tagged v1.0.0 .
16-18	fix(security):, fix(sequence):, fix(ci):	The three defects reported in section 3.6.
19	docs: record the 1.0.1 concurrency fixes	Tagged v1.0.1 .

4.4 Workflows (CI/CD)

`ci.yml` runs on every push and pull request. It builds on a **matrix of JDK 17 and JDK 21** - the project targets 17 for portability, and the matrix proves it also runs on a newer runtime - then publishes test and coverage reports as artefacts, packages both executable jars, and boots the packaged server to confirm it answers its health check. A separate job runs a Trivy vulnerability scan.

``release.yml`` is triggered only by a tag matching `v*.*.*`. It rebuilds from the tagged commit and creates a GitHub Release carrying both jars and that version's changelog section, so a release can never be published from uncommitted work. Together the two mean the state of `main` is always known, and producing a release is one command rather than a checklist somebody can get wrong at midnight.

4.5 Evaluation

One instance proves the approach worked. When the concurrency defects in section 3.6 were found **after** `v1.0.0` was tagged, the fix was three focused commits and a `v1.0.1` tag - and because the history was granular and the messages conventional, it was possible to see exactly which commit had introduced the login bookkeeping and what it had been trying to achieve. That is the practical value of a disciplined history: the ability to answer “why is this like this?” months later.

Two honest limitations. The brief asks for versions updated **each day**; these increments are grouped by architectural layer rather than by calendar day, which is easier to follow but does not literally show daily activity. And as a single-developer project there were no pull requests to review and no merge conflicts, so the branching model was never tested under the conditions it exists for.

References

- Beck, K., 2003. *Test-Driven Development: By Example*. Boston: Addison-Wesley.
- Bloch, J., 2018. *Effective Java*. 3rd ed. Boston: Addison-Wesley.
- Driessen, V., 2010. *A successful Git branching model*. [Online] Available at: <https://nvie.com/posts/a-successful-git-branching-model/> [Accessed 28 07 2026].
- Evans, E., 2004. *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Boston: Addison-Wesley.
- Fowler, M., 2002. *Patterns of Enterprise Application Architecture*. Boston: Addison-Wesley.
- Fowler, M., 2003. *UML Distilled: A Brief Guide to the Standard Object Modeling Language*. 3rd ed. Boston: Addison-Wesley.
- Freeman, S. and Pryce, N., 2009. *Growing Object-Oriented Software, Guided by Tests*. Boston: Addison-Wesley.
- Gamma, E., Helm, R., Johnson, R. and Vlissides, J., 1995. *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA: Addison-Wesley.
- GitHub, 2026. *GitHub Actions documentation*. [Online] Available at: <https://docs.github.com/en/actions> [Accessed 02 08 2026].
- Martin, R. C., 2008. *Clean Code: A Handbook of Agile Software Craftsmanship*. Upper Saddle River, NJ: Prentice Hall.
- Object Management Group, 2017. *OMG Unified Modeling Language (OMG UML), Version 2.5.1*. [Online] Available at: <https://www.omg.org/spec/UML/2.5.1/> [Accessed 20 07 2026].
- Oracle, 2024. *Java Platform, Standard Edition 17 Documentation*. [Online] Available at: <https://docs.oracle.com/en/java/javase/17/> [Accessed 15 07 2026].
- OWASP Foundation, 2021. *OWASP Top Ten Web Application Security Risks*. [Online] Available at: <https://owasp.org/www-project-top-ten/> [Accessed 25 07 2026].

Preston-Werner, T., 2013. *Semantic Versioning 2.0.0*. [Online] Available at: <https://semver.org/> [Accessed 28 07 2026].

Red Gate Software, 2024. *Flyway Documentation*. [Online] Available at: <https://documentation.red-gate.com/flyway> [Accessed 18 07 2026].

VMware, 2025. *Spring Boot Reference Documentation, version 3.5*. [Online] Available at: <https://docs.spring.io/spring-boot/docs/current/reference/html/> [Accessed 10 07 2026].

VMware, 2025. *Spring Security Reference*. [Online] Available at: <https://docs.spring.io/spring-security/reference/> [Accessed 22 07 2026].